```java
package com.feel.agentcore.action;

import java.util.Objects;
import java.util.UUID;

public class Action {
   private final String id;
   private final ActionType type;
   private final float energyCost;
   private final float boredomEffect;
   private final float noveltyValue;

   public Action(ActionType type, float energyCost, float
boredomEffect, float noveltyValue) {
       this.id = UUID.randomUUID().toString();
       this.type = type;
       this.energyCost = energyCost;
       this.boredomEffect = boredomEffect;
       this.noveltyValue = noveltyValue;
   }

   public String getId() { return id; }
   public ActionType getType() { return type; }
   public float getEnergyCost() { return energyCost; }
   public float getBoredomEffect() { return boredomEffect; }
   public float getNoveltyValue() { return noveltyValue; }

   public Action copy() {
       return new Action(type, energyCost, boredomEffect,
noveltyValue);
   }

   @Override
   public boolean equals(Object o) {
       if (this == o) return true;
       if (o == null || getClass() != o.getClass()) return
false;
       Action action = (Action) o;
       return id.equals(action.id);
   }
```

```
    @Override
    public int hashCode() {
        return Objects.hash(id);
    }

    @Override
    public String toString() {
        return String.format("%s", type);
    }
}
```

```
package com.feel.agentcore.action;

public enum ActionType {
    IDLE,
    EXPLORE,
    REPEAT_LAST,
    REST
}
```

```
package com.feel.agentcore.core;

import com.feel.agentcore.mind.AgentMind;
import com.feel.agentcore.mind.Feedback;
import com.feel.agentcore.decision.DecisionCore;
import com.feel.agentcore.world.World;
import com.feel.agentcore.action.Action;
import com.feel.agentcore.action.ActionType;
import com.feel.agentcore.persistence.StateManager;
import com.feel.agentcore.observer.ObservationLogger;
import java.util.concurrent.ScheduledExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;

public class CognitiveLoop {
    private final long baseTickIntervalMs = 500;
    private float timeScale = 1.0f;
    private boolean running = false;
```

```java
private final AgentMind agentMind;
private final DecisionCore decisionCore;
private final World world;
private final StateManager stateManager;
private final ObservationLogger logger;

private ScheduledExecutorService scheduler;
private long internalTime = 0;

public CognitiveLoop() {
    this.agentMind = new AgentMind();
    this.decisionCore = new DecisionCore();
    this.world = new World();
    this.stateManager = StateManager.getInstance();
    this.logger = new ObservationLogger();
}

public void start() {
    running = true;
    scheduler = Executors.newSingleThreadScheduledExecutor();
    scheduler.scheduleAtFixedRate(
            this::tick,
            0,
            (long)(baseTickIntervalMs / timeScale),
            TimeUnit.MILLISECONDS
    );
}

public void pause() {
    running = false;
    if (scheduler != null) {
        scheduler.shutdown();
    }
}

public void togglePause() {
    if (running) {
        pause();
    } else {
        start();
```

```
        }
    }

    public void stop() {
        pause();
        if (stateManager != null) {
            stateManager.save(agentMind, world, internalTime);
        }
        if (logger != null) {
            logger.close();
        }
    }

    public void setTimeScale(float scale) {
        this.timeScale = scale;
        if (running) {
            pause();
            start();
        }
    }

    private void tick() {
        if (!running) return;

        float deltaTime = 1.0f * timeScale;

        // Sense environment
        world.sense();

        // Update internal states
        agentMind.updateInternalState(deltaTime);
        agentMind.applyInertia(deltaTime);

        // Compute decision
        float pressure = decisionCore.computePressure(agentMind);
        float threshold =
decisionCore.computeThreshold(agentMind);

        // Decide and act
        if (pressure >= threshold && agentMind.canAct()) {
```

```
            Action selectedAction =
decisionCore.selectAction(agentMind, world);
            Feedback feedback =
world.executeAction(selectedAction);

            // Log action
            logger.logAction(internalTime, agentMind,
selectedAction.getType(), feedback, agentMind);

            // Update mind with outcome
            agentMind.registerOutcome(selectedAction.getType(),
feedback);
        } else {
            // Idle tick
            if (agentMind.getEnergy() < 0.1f) {
                // Force rest when energy is critically low
                agentMind.registerOutcome(ActionType.REST,
Feedback.POSITIVE);
            } else {
                agentMind.registerOutcome(ActionType.IDLE,
Feedback.NEUTRAL);
            }
        }

        // Log observation
        logger.log(createSnapshot());

        internalTime++;
    }

    private LogSnapshot createSnapshot() {
        return new LogSnapshot(
                internalTime,
                agentMind,
                world.getCurrentState()
        );
    }

    // Getters
    public AgentMind getAgentMind() { return agentMind; }
    public float getTimeScale() { return timeScale; }
```

```
   public World getWorld() { return world; }

   // Inner class for snapshots
   public static class LogSnapshot {
       public final long time;
       public final AgentMind mind;
       public final World.State worldState;

       public LogSnapshot(long time, AgentMind mind, World.State
worldState) {
           this.time = time;
           this.mind = mind;
           this.worldState = worldState;
       }
   }
}
```

```
package com.feel.agentcore.decision;

import com.feel.agentcore.mind.AgentMind;
import com.feel.agentcore.world.World;
import com.feel.agentcore.action.Action;
import com.feel.agentcore.action.ActionType;
import com.feel.agentcore.util.Noise;
import java.util.*;

public class DecisionCore {
   // Adjusted weights - boredom and curiosity now matter MORE
   private final float BOREDOM_WEIGHT = 0.5f;      // Increased
   private final float CURIOSITY_WEIGHT = 0.4f;    // Increased
   private final float FRUSTRATION_WEIGHT = 0.2f;  // Decreased

   // LOWER threshold = easier to act
   private final float BASE_THRESHOLD = 0.35f;      // Lowered
from 0.5

   private final float MIN_ENERGY_FOR_ACTION = 0.08f;

   private transient Random random;
   private ActionType lastSelectedAction = null;
```

```java
    private int sameActionCount = 0;

    public DecisionCore() {
        this.random = new Random();
    }

    public void initAfterLoad() {
        if (random == null) {
            this.random = new Random();
        }
    }

    public float computePressure(AgentMind mind) {
        float boredom = mind.getBoredom();
        float curiosity = mind.getCuriosity();
        float frustration = mind.getFrustration();
        float energy = mind.getEnergy();

        // Pressure calculation - more weight on boredom and
curiosity
        float pressure = (boredom * BOREDOM_WEIGHT) +
                (curiosity * CURIOSITY_WEIGHT) +
                (frustration * FRUSTRATION_WEIGHT);

        // Energy effect - less punishing at low energy
        if (energy < 0.15f) {
            pressure = pressure * 0.6f;  // Less reduction
        } else {
            pressure = pressure * Math.min(1.2f, energy * 1.8f);
// Boost when energy is good
        }

        pressure += Noise.smallGaussian();
        return Math.max(0, Math.min(1, pressure));
    }

    public float computeThreshold(AgentMind mind) {
        float energy = mind.getEnergy();
        float confidence = mind.getConfidence();
        float decisionInertia = mind.getDecisionInertia();
```

```
        float threshold = BASE_THRESHOLD;

        // Energy effect - less impact
        if (energy < 0.15f) {
            threshold += 0.2f;  // Smaller increase
        } else {
            threshold += (1.0f - energy) * 0.1f; // Smaller
effect
        }

        threshold += (1.0f - confidence) * 0.15f;
        threshold += decisionInertia * 0.15f;
        threshold += Noise.smallGaussian() * 0.1f;

        return Math.max(0.15f, Math.min(0.8f, threshold));
    }

    public Action selectAction(AgentMind mind, World world) {
        List<Action> availableActions =
world.getAvailableActions();

        if (mind.getEnergy() < 0.08f) {
            for (Action action : availableActions) {
                if (action.getType() == ActionType.REST) {
                    return action;
                }
            }
        }

        if (availableActions.isEmpty()) {
            return createIdleAction();
        }

        Map<Action, Float> probabilities = new HashMap<>();
        float totalProbability = 0;

        for (Action action : availableActions) {
            float probability =
calculateActionProbability(action, mind);

            // Anti-repetition - stronger penalty for repetition
```

```
            if (action.getType() == lastSelectedAction) {
                probability *= (1.0f - Math.min(0.7f,
sameActionCount * 0.15f));
            }

            // Boost for EXPLORE when bored or curious
            if (action.getType() == ActionType.EXPLORE) {
                if (mind.getBoredom() > 0.5f) {
                    probability *= 1.5f;
                }
                if (mind.getCuriosity() > 0.6f) {
                    probability *= 1.5f;
                }
            }

            // Penalize IDLE when very bored
            if (action.getType() == ActionType.IDLE &&
mind.getBoredom() > 0.7f) {
                probability *= 0.5f;
            }

            // Boost REST when energy is low
            if (action.getType() == ActionType.REST &&
mind.getEnergy() < 0.3f) {
                probability *= (1.0f + (0.3f - mind.getEnergy())
* 3.0f);
            }

            probabilities.put(action, probability);
            totalProbability += probability;
        }

        if (totalProbability > 0) {
            for (Map.Entry<Action, Float> entry :
probabilities.entrySet()) {
                probabilities.put(entry.getKey(),
entry.getValue() / totalProbability);
            }
        }

        Action selected = selectByProbability(probabilities);
```

```
        if (selected.getType() == lastSelectedAction) {
            sameActionCount++;
        } else {
            sameActionCount = 0;
            lastSelectedAction = selected.getType();
        }

        return selected;
    }

    private float calculateActionProbability(Action action,
AgentMind mind) {
        float probability = 1.5f; // Higher base probability

        // Strong novelty bias
        if (action.getNoveltyValue() > 0.3f) {
            probability *= (1.0f + mind.getNoveltyBias() * 1.5f);
        }

        // Habit strength - less influence
        float habitStrength =
mind.getHabitStrength(action.getType());
        probability *= (1.0f + habitStrength *
mind.getPersistence() * 0.5f);

        // Emotional memory - strong influence
        Float emotionalScore =
mind.getEmotionalMemory(action.getType());
        if (emotionalScore != null) {
            probability *= (1.0f + emotionalScore *
mind.getSensitivityToStimuli() * 1.2f);
        }

        // Energy cost - important but not prohibitive
        float energyCost = action.getEnergyCost();
        float availableEnergy = mind.getEnergy();

        if (energyCost > 0) {
            if (availableEnergy < energyCost * 0.8f) {
                probability *= 0.3f;
```

```
            } else {
                probability *= (1.0f - (energyCost /
availableEnergy) * 0.3f);
            }
        } else { // Energy gain actions (like REST)
            if (availableEnergy < 0.4f) {
                probability *= 3.0f;
            }
        }

        // Add randomness
        probability += Noise.smallGaussian() * 0.3f;

        return Math.max(0.3f, probability);
    }

    private Action selectByProbability(Map<Action, Float>
probabilities) {
        double randomValue = random.nextDouble();
        double cumulativeProbability = 0.0;

        List<Map.Entry<Action, Float>> sorted = new
ArrayList<>(probabilities.entrySet());
        sorted.sort((a, b) -> Float.compare(b.getValue(),
a.getValue()));

        for (Map.Entry<Action, Float> entry : sorted) {
            cumulativeProbability += entry.getValue();
            if (randomValue <= cumulativeProbability) {
                return entry.getKey();
            }
        }

        return sorted.get(0).getKey();
    }

    private Action createIdleAction() {
        return new Action(ActionType.IDLE, 0.1f, 0.05f, 0.0f);
    }

    public void resetRepetitionTracking() {
```

```
        lastSelectedAction = null;
        sameActionCount = 0;
    }
}
```

```
package com.feel.agentcore.decision;

import com.feel.agentcore.mind.AgentMind;

public class DecisionCoreTest {
    public static void main(String[] args) {
        System.out.println("🧪 Testing DecisionCore\n");

        AgentMind mind = new AgentMind();
        DecisionCore decisionCore = new DecisionCore();

        // Test 1: Calculate pressure with default state
        float pressure = decisionCore.computePressure(mind);
        System.out.println("Default state pressure: " +
pressure);

        // Test 2: Modify mind state and see pressure change
        System.out.println("\nSimulating high boredom...");
        // We'll need to add a setter method to AgentMind
temporarily
        // Or modify AgentMind to have package-private access for
testing

        // Test 3: Calculate threshold
        float threshold = decisionCore.computeThreshold(mind);
        System.out.println("Current threshold: " + threshold);

        // Test 4: Check if action would be taken
        System.out.println("\nWould act? " + (pressure >=
threshold ? "YES" : "NO"));

        // Run multiple times to see probabilistic nature
        System.out.println("\nRunning 10 decision cycles:");
        for (int i = 0; i < 10; i++) {
            pressure = decisionCore.computePressure(mind);
```

```
            threshold = decisionCore.computeThreshold(mind);
            System.out.printf("Cycle %2d: pressure=%.3f,
threshold=%.3f, act=%s%n",
                    i+1, pressure, threshold, (pressure >=
threshold ? "✓" : "✗"));
        }
    }
}
```

```
package com.feel.agentcore.mind;

import com.feel.agentcore.action.ActionType;
import java.util.*;

public class AgentMind {
    // Internal dynamic state (volatile, changes fast)
    private float energy;        // 0-1
    private float boredom;       // 0-1
    private float curiosity;     // 0-1
    private float frustration;   // 0-1
    private float confidence;    // 0-1

    // Personality parameters (slow-changing)
    private float riskTolerance;     // 0-1
    private float noveltyBias;       // 0-1
    private float persistence;       // 0-1
    private float sensitivityToStimuli; // 0-1
    private float decisionInertia;   // 0-1

    // Memory structures
    private Queue<ActionRecord> recentActions;
    private Map<ActionType, Float> emotionalMemory;
    private Map<ActionType, HabitData> habitTracker;

    // Constants
    private static final float ENERGY_DRAIN_RATE = 0.015f;
    private static final float ENERGY_RESTORE_RATE = 0.06f;
    private static final float BOREDOM_INCREASE_RATE = 0.04f;
    private static final float BOREDOM_DECREASE_RATE = 0.12f;
    private static final float CURIOSITY_BUILD_RATE = 0.05f;
```

```java
    private static final float CURIOSITY_SATISFY_RATE = 0.15f;
    private static final float FRUSTRATION_DECAY_RATE = 0.02f;
    private static final float FRUSTRATION_BUILD_RATE = 0.08f;
    private static final float MIN_ENERGY_TO_ACT = 0.08f;

    // Confidence limits and dynamics
    private static final float MAX_CONFIDENCE = 0.85f;
    private static final float MIN_CONFIDENCE = 0.15f;
    private static final float CONFIDENCE_DECAY_RATE = 0.01f;
    private static final float CONFIDENCE_BOOST_RATE = 0.04f;
    private static final float CONFIDENCE_DROP_RATE = 0.06f;

    public AgentMind() {
        // Start in a balanced state
        this.energy = 0.8f;
        this.boredom = 0.3f;
        this.curiosity = 0.6f;
        this.frustration = 0.2f;
        this.confidence = 0.6f;

        this.riskTolerance = 0.6f;
        this.noveltyBias = 0.7f;
        this.persistence = 0.4f;
        this.sensitivityToStimuli = 0.6f;
        this.decisionInertia = 0.3f;

        this.recentActions = new LinkedList<>();
        this.emotionalMemory = new HashMap<>();
        this.habitTracker = new HashMap<>();
    }

    public void updateInternalState(float deltaTime) {
        // Energy dynamics
        if (energy < 0.1f) {
            energy += ENERGY_RESTORE_RATE * deltaTime * 2;
        } else {
            energy -= ENERGY_DRAIN_RATE * deltaTime;
        }

        // Boredom increases naturally
        boredom += BOREDOM_INCREASE_RATE * deltaTime;
```

```
        // Curiosity builds from boredom
        if (boredom > 0.2f) {
            curiosity += CURIOSITY_BUILD_RATE * boredom *
deltaTime;
        }

        // Frustration naturally decays, but never reaches 0
        frustration -= FRUSTRATION_DECAY_RATE * deltaTime;
        if (frustration < 0.05f) {
            frustration = 0.05f;
        }

        // Confidence slowly decays toward baseline if not
boosted
        confidence -= CONFIDENCE_DECAY_RATE * deltaTime;

        // Add some random fluctuation
        confidence += (Math.random() - 0.5) * 0.02f * deltaTime;

        clampAllStates();
    }

    /**
     * Apply inertia to smooth out changes and prevent sudden
mood swings
     * This method is called from CognitiveLoop
     */
    public void applyInertia(float deltaTime) {
        // Inertia slows down rate of change based on
decisionInertia personality trait
        float inertiaFactor = decisionInertia * deltaTime;

        // Store current values before applying inertia
        float oldEnergy = energy;
        float oldBoredom = boredom;
        float oldCuriosity = curiosity;
        float oldFrustration = frustration;
        float oldConfidence = confidence;
```

```
        // Apply inertia - values tend to stay closer to their
previous values
        // Higher decisionInertia = more resistance to change
        energy = oldEnergy * (1 - inertiaFactor) + energy *
inertiaFactor;
        boredom = oldBoredom * (1 - inertiaFactor) + boredom *
inertiaFactor;
        curiosity = oldCuriosity * (1 - inertiaFactor) +
curiosity * inertiaFactor;
        frustration = oldFrustration * (1 - inertiaFactor) +
frustration * inertiaFactor;
        confidence = oldConfidence * (1 - inertiaFactor) +
confidence * inertiaFactor;

        // Ensure we still respect bounds after inertia
        clampAllStates();
    }

    public void registerOutcome(ActionType action, Feedback
feedback) {
        float emotionalWeight = feedback == Feedback.POSITIVE ?
0.8f :
                feedback == Feedback.NEGATIVE ? -0.7f : 0.1f;
        recordAction(action, emotionalWeight);

        switch (feedback) {
            case POSITIVE:
                confidence += CONFIDENCE_BOOST_RATE;
                frustration = Math.max(0.05f, frustration -
0.04f);
                curiosity -= CURIOSITY_SATISFY_RATE * 0.8f;
                boredom -= BOREDOM_DECREASE_RATE * 0.9f;
                break;

            case NEGATIVE:
                confidence -= CONFIDENCE_DROP_RATE;
                frustration += FRUSTRATION_BUILD_RATE;
                curiosity += 0.03f;
                boredom -= 0.03f;
                break;
```

```
            case NEUTRAL:
                confidence -= 0.01f;
                frustration += 0.01f;
                curiosity += 0.01f;
                boredom -= 0.02f;
                break;
        }

        // Energy effects
        switch (action) {
            case REST:
                energy += 0.15f;
                boredom += 0.02f;
                break;

            case EXPLORE:
                energy -= 0.10f;
                curiosity -= 0.10f;
                boredom -= 0.12f;
                break;

            case REPEAT_LAST:
                energy -= 0.05f;
                curiosity -= 0.02f;
                boredom -= 0.04f;
                break;

            case IDLE:
                energy -= 0.02f;
                boredom += 0.06f;
                curiosity += 0.02f;
                break;
        }

        clampAllStates();
        updateEmotionalMemory(action, feedback);
        updateHabitTracker(action);
    }

    public MoodSnapshot getDerivedMood() {
        float valence = confidence - frustration;
```

```
        float arousal = curiosity - boredom;

        return new MoodSnapshot(valence, arousal);
    }

    public float getHabitStrength(ActionType action) {
        HabitData data = habitTracker.get(action);
        if (data == null) {
            return 0.0f;
        }

        float frequencyStrength = Math.min(1.0f, data.frequency /
10.0f);
        long timeSinceLastPerformed = System.currentTimeMillis()
- data.lastPerformed;
        float recencyBonus = (float)
Math.exp(-timeSinceLastPerformed / 60000.0);

        return frequencyStrength * 0.6f + recencyBonus * 0.4f;
    }

    public Float getEmotionalMemory(ActionType action) {
        return emotionalMemory.get(action);
    }

    public void recordAction(ActionType action, float
emotionalWeight) {
        ActionRecord record = new ActionRecord();
        record.action = action;
        record.timestamp = System.currentTimeMillis();
        record.emotionalWeight = emotionalWeight;

        recentActions.add(record);

        while (recentActions.size() > 20) {
            recentActions.poll();
        }

        updateHabitTracker(action);
    }
```

```java
    public ActionType getLastAction() {
        ActionRecord last = recentActions.peek();
        return last != null ? last.action : null;
    }

    public int getActionFrequency(ActionType action) {
        HabitData data = habitTracker.get(action);
        return data != null ? data.frequency : 0;
    }

    public float getDecisionPressure() {
        float basePressure = (boredom * 0.4f) + (curiosity *
0.3f) + (frustration * 0.2f);

        // Confidence affects pressure - low confidence = less
pressure to act
        basePressure *= (0.5f + confidence * 0.5f);

        if (energy < MIN_ENERGY_TO_ACT) {
            basePressure *= (energy / MIN_ENERGY_TO_ACT);
        }

        return basePressure;
    }

    public boolean canAct() {
        return energy >= MIN_ENERGY_TO_ACT;
    }

    public boolean isStuck() {
        return energy < 0.05f && boredom > 0.95f;
    }

    public List<ActionRecord> getRecentActionsList() {
        return new ArrayList<>(recentActions);
    }

    private void clampAllStates() {
        energy = clamp(energy, 0, 1);
        boredom = clamp(boredom, 0, 1);
        curiosity = clamp(curiosity, 0, 1);
```

```
        frustration = clamp(frustration, 0.05f, 1);
        confidence = clamp(confidence, MIN_CONFIDENCE,
MAX_CONFIDENCE);

        riskTolerance = clamp(riskTolerance, 0, 1);
        noveltyBias = clamp(noveltyBias, 0, 1);
        persistence = clamp(persistence, 0, 1);
        sensitivityToStimuli = clamp(sensitivityToStimuli, 0, 1);
        decisionInertia = clamp(decisionInertia, 0, 1);
    }

    private float clamp(float value, float min, float max) {
        return Math.max(min, Math.min(max, value));
    }

    private void updateEmotionalMemory(ActionType action,
Feedback feedback) {
        float emotionalScore = feedback == Feedback.POSITIVE ?
0.8f :
                feedback == Feedback.NEGATIVE ? -0.8f : 0.1f;
        Float existing = emotionalMemory.get(action);
        if (existing != null) {
            emotionalScore = existing * 0.7f + emotionalScore *
0.3f;
        }
        emotionalMemory.put(action, emotionalScore);
    }

    private void updateHabitTracker(ActionType action) {
        HabitData data = habitTracker.getOrDefault(action, new
HabitData());
        data.incrementCount();
        data.updateRecency();
        habitTracker.put(action, data);
    }

    // Getters
    public float getEnergy() { return energy; }
    public float getBoredom() { return boredom; }
    public float getCuriosity() { return curiosity; }
```

```
   public float getFrustration() { return frustration; }
   public float getConfidence() { return confidence; }
   public float getRiskTolerance() { return riskTolerance; }
   public float getNoveltyBias() { return noveltyBias; }
   public float getPersistence() { return persistence; }
   public float getSensitivityToStimuli() { return
sensitivityToStimuli; }
   public float getDecisionInertia() { return decisionInertia; }
   public Queue<ActionRecord> getRecentActions() { return
recentActions; }
   public Map<ActionType, Float> getEmotionalMemory() { return
emotionalMemory; }
   public Map<ActionType, HabitData> getHabitTracker() { return
habitTracker; }

   // Setters
   public void setEnergy(float energy) { this.energy =
clamp(energy, 0, 1); }
   public void setBoredom(float boredom) { this.boredom =
clamp(boredom, 0, 1); }
   public void setCuriosity(float curiosity) { this.curiosity =
clamp(curiosity, 0, 1); }
   public void setFrustration(float frustration) {
this.frustration = clamp(frustration, 0.05f, 1); }
   public void setConfidence(float confidence) { this.confidence
= clamp(confidence, MIN_CONFIDENCE, MAX_CONFIDENCE); }
   public void setRiskTolerance(float riskTolerance) {
this.riskTolerance = clamp(riskTolerance, 0, 1); }
   public void setNoveltyBias(float noveltyBias) {
this.noveltyBias = clamp(noveltyBias, 0, 1); }
   public void setPersistence(float persistence) {
this.persistence = clamp(persistence, 0, 1); }
   public void setSensitivityToStimuli(float
sensitivityToStimuli) { this.sensitivityToStimuli =
clamp(sensitivityToStimuli, 0, 1); }
   public void setDecisionInertia(float decisionInertia) {
this.decisionInertia = clamp(decisionInertia, 0, 1); }
   public void setRecentActions(Queue<ActionRecord>
recentActions) { this.recentActions = recentActions; }
   public void setEmotionalMemory(Map<ActionType, Float>
emotionalMemory) { this.emotionalMemory = emotionalMemory; }
```

```
    public void setHabitTracker(Map<ActionType, HabitData>
habitTracker) { this.habitTracker = habitTracker; }

    // Inner classes
    public static class ActionRecord {
        public ActionType action;
        public long timestamp;
        public float emotionalWeight;

        @Override
        public String toString() {
            return String.format("%s", action);
        }
    }

    public static class HabitData {
        public int frequency = 0;
        public long lastPerformed = 0;

        public void incrementCount() { frequency++; }
        public void updateRecency() { lastPerformed =
System.currentTimeMillis(); }
    }

    public static class MoodSnapshot {
        public final float valence;
        public final float arousal;

        public MoodSnapshot(float valence, float arousal) {
            this.valence = valence;
            this.arousal = arousal;
        }

        public String getInterpretation() {
            if (valence > 0.3f && arousal > 0.3f) return "excited
curiosity";
            if (valence > 0.3f && arousal < -0.3f) return "calm
contentment";
            if (valence < -0.3f && arousal > 0.3f) return
"agitated frustration";
```

```
            if (valence < -0.3f && arousal < -0.3f) return
"gloomy withdrawal";
            if (valence > 0.3f) return "positive";
            if (valence < -0.3f) return "negative";
            return "neutral";
        }
    }
}
```

```
package com.feel.agentcore.mind;

public enum Feedback {
    POSITIVE,
    NEGATIVE,
    NEUTRAL
}
```

```
package com.feel.agentcore.observer;

import com.feel.agentcore.mind.AgentMind;
import com.feel.agentcore.mind.Feedback;
import com.feel.agentcore.action.ActionType;
import com.feel.agentcore.core.CognitiveLoop;
import java.io.*;
import java.nio.file.*;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.*;

public class ObservationLogger {
    private final String LOG_DIR = "logs";
    private final String LOG_FILE;
    private BufferedWriter writer;
    private final List<LogEntry> recentEntries;
    private final int MAX_RECENT_ENTRIES = 100;

    public ObservationLogger() {
        try {
            Files.createDirectories(Paths.get(LOG_DIR));
```

```
        } catch (IOException e) {
            System.err.println("Failed to create logs directory:
" + e.getMessage());
        }

        String timestamp =
LocalDateTime.now().format(DateTimeFormatter.ofPattern("yyyyMMdd
_HHmmss"));
        LOG_FILE = LOG_DIR + "/agent_log_" + timestamp + ".txt";

        this.recentEntries = new ArrayList<>();
        initLogFile();
    }

    private void initLogFile() {
        try {
            writer = new BufferedWriter(new FileWriter(LOG_FILE,
true));
            writeHeader();
        } catch (IOException e) {
            System.err.println("Failed to initialize log file: "
+ e.getMessage());
        }
    }

    private void writeHeader() {
        String header = "=".repeat(100) + "\n";
        header += "AGENT OBSERVATION LOG\n";
        header += "Started: " + LocalDateTime.now() + "\n";
        header += "=".repeat(100) + "\n\n";

        try {
            writer.write(header);
            writer.flush();
        } catch (IOException e) {
            System.err.println("Failed to write header: " +
e.getMessage());
        }
    }

    public void log(Object snapshot) {
```

```
        if (snapshot instanceof CognitiveLoop.LogSnapshot) {
            logLoopSnapshot((CognitiveLoop.LogSnapshot)
snapshot);
        }
    }

    private void logLoopSnapshot(CognitiveLoop.LogSnapshot
snapshot) {
        AgentMind mind = snapshot.mind;

        LogEntry entry = new LogEntry(
                snapshot.time,
                mind.getEnergy(),
                mind.getBoredom(),
                mind.getCuriosity(),
                mind.getFrustration(),
                mind.getConfidence(),
                mind.getDerivedMood(),
                null,
                null
        );

        String logLine = formatLogEntry(entry);

        try {
            writer.write(logLine + "\n");
            writer.flush();
        } catch (IOException e) {
            System.err.println("Failed to write log entry: " +
e.getMessage());
        }

        recentEntries.add(entry);
        if (recentEntries.size() > MAX_RECENT_ENTRIES) {
            recentEntries.remove(0);
        }
    }

    public void logAction(long time, AgentMind beforeMind,
ActionType action,
```

```
                            Feedback feedback, AgentMind afterMind)
{

        LogEntry entry = new LogEntry(
                time,
                beforeMind.getEnergy(),
                beforeMind.getBoredom(),
                beforeMind.getCuriosity(),
                beforeMind.getFrustration(),
                beforeMind.getConfidence(),
                beforeMind.getDerivedMood(),
                action,
                feedback
        );

        String logLine = formatLogEntry(entry);

        try {
            writer.write(logLine + "\n");
            writer.flush();
        } catch (IOException e) {
            System.err.println("Failed to write log entry: " +
e.getMessage());
        }

        recentEntries.add(entry);
        if (recentEntries.size() > MAX_RECENT_ENTRIES) {
            recentEntries.remove(0);
        }
    }

    private String formatLogEntry(LogEntry entry) {
        StringBuilder sb = new StringBuilder();

        sb.append(String.format("[T=%04d] ", entry.time));
        sb.append(String.format("E:%.2f B:%.2f C:%.2f F:%.2f
K:%.2f | ",
                entry.energy, entry.boredom, entry.curiosity,
                entry.frustration, entry.confidence));
```

```
        sb.append(String.format("M:(v=%.2f,a=%.2f)",
entry.mood.valence, entry.mood.arousal));

        if (entry.action != null) {
            sb.append(String.format(" | %s → %s", entry.action,
entry.feedback));
        }

        return sb.toString();
    }

    public List<LogEntry> getRecentEntries() {
        return new ArrayList<>(recentEntries);
    }

    public void close() {
        try {
            if (writer != null) {
                writer.write("\n" + "=".repeat(100) + "\n");
                writer.write("LOG ENDED: " + LocalDateTime.now()
+ "\n");
                writer.close();
            }
        } catch (IOException e) {
            System.err.println("Failed to close log file: " +
e.getMessage());
        }
    }

    public static class LogEntry {
        public final long time;
        public final float energy;
        public final float boredom;
        public final float curiosity;
        public final float frustration;
        public final float confidence;
        public final AgentMind.MoodSnapshot mood;
        public final ActionType action;
        public final Feedback feedback;
```

```
        public LogEntry(long time, float energy, float boredom,
float curiosity,
                        float frustration, float confidence,
                        AgentMind.MoodSnapshot mood, ActionType
action, Feedback feedback) {
            this.time = time;
            this.energy = energy;
            this.boredom = boredom;
            this.curiosity = curiosity;
            this.frustration = frustration;
            this.confidence = confidence;
            this.mood = mood;
            this.action = action;
            this.feedback = feedback;
        }
    }
}
```

```
package com.feel.agentcore.persistence;

import com.google.gson.*;
import java.lang.reflect.Type;
import java.util.Random;

public class RandomAdapter implements JsonSerializer<Random>,
JsonDeserializer<Random> {

    @Override
    public JsonElement serialize(Random src, Type typeOfSrc,
JsonSerializationContext context) {
        JsonObject object = new JsonObject();
        // Just store a dummy value - Random state isn't critical
for persistence
        object.addProperty("dummy", 0);
        return object;
    }

    @Override
    public Random deserialize(JsonElement json, Type typeOfT,
JsonDeserializationContext context) {
```

```
        // Return a new Random instance when loading
        return new Random();
    }
}
```

```
package com.feel.agentcore.persistence;

import com.feel.agentcore.mind.AgentMind;
import com.feel.agentcore.world.World;
import com.google.gson.*;
import java.io.*;
import java.lang.reflect.Modifier;
import java.nio.file.*;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.Random;

public class StateManager {
    private static StateManager instance;
    private final String SAVE_DIR = "saves";
    private final String AUTOSAVE_FILE = "saves/autosave.json";
    private final Gson gson;

    private StateManager() {
        try {
            Files.createDirectories(Paths.get(SAVE_DIR));
        } catch (IOException e) {
            System.err.println("Failed to create saves directory:
" + e.getMessage());
        }

        // Create Gson with custom adapters and configuration
        this.gson = new GsonBuilder()
                .setPrettyPrinting()
                .excludeFieldsWithModifiers(Modifier.TRANSIENT)
// Skip transient fields
                .registerTypeAdapter(Random.class, new
RandomAdapter())
                .create();
    }
```

```java
    public static StateManager getInstance() {
        if (instance == null) {
            instance = new StateManager();
        }
        return instance;
    }

    public void save(AgentMind mind, World world, long
internalTime) {
        SaveState saveState = new SaveState(mind, world,
internalTime, LocalDateTime.now());

        try (Writer writer = new FileWriter(AUTOSAVE_FILE)) {
            gson.toJson(saveState, writer);
            System.out.println("💾 Agent state saved
successfully");
        } catch (IOException e) {
            System.err.println("Failed to save agent state: " +
e.getMessage());
            e.printStackTrace();
        }
    }

    public LoadedState load() {
        File file = new File(AUTOSAVE_FILE);
        if (!file.exists()) {
            return null;
        }

        try (Reader reader = new FileReader(file)) {
            SaveState saveState = gson.fromJson(reader,
SaveState.class);
            return new LoadedState(saveState);
        } catch (IOException e) {
            System.err.println("Failed to load agent state: " +
e.getMessage());
            return null;
        }
    }
```

```java
    private static class SaveState {
        final AgentMind mind;
        final World world;
        final long internalTime;
        final LocalDateTime saveTime;

        SaveState(AgentMind mind, World world, long internalTime,
LocalDateTime saveTime) {
            this.mind = mind;
            this.world = world;
            this.internalTime = internalTime;
            this.saveTime = saveTime;
        }
    }

    public static class LoadedState {
        public final AgentMind mind;
        public final World world;
        public final long internalTime;
        public final LocalDateTime saveTime;

        LoadedState(SaveState saveState) {
            this.mind = saveState.mind;
            this.world = saveState.world;
            this.internalTime = saveState.internalTime;
            this.saveTime = saveState.saveTime;
        }
    }
}
```

```java
package com.feel.agentcore.util;

public class Clamp {
    /**
     * Clamp a float value between min and max
     */
    public static float clamp(float value, float min, float max)
{
        return Math.max(min, Math.min(max, value));
    }
```

```
    /**
     * Clamp a double value between min and max
     */
    public static double clamp(double value, double min, double
max) {
        return Math.max(min, Math.min(max, value));
    }

    /**
     * Clamp an int value between min and max
     */
    public static int clamp(int value, int min, int max) {
        return Math.max(min, Math.min(max, value));
    }

    /**
     * Normalize a value to 0-1 range
     */
    public static float normalize(float value, float min, float
max) {
        return (value - min) / (max - min);
    }
}
```

```
package com.feel.agentcore.util;

import java.util.Random;

public class Noise {
    private static final Random random = new Random();

    public static float smallGaussian() {
        return (float) random.nextGaussian() * 0.15f;  //
Increased from 0.1
    }

    public static float mediumGaussian() {
        return (float) random.nextGaussian() * 0.3f;   //
Increased from 0.2
```

```
    }

    public static float uniform(float range) {
        return (random.nextFloat() * 2 - 1) * range;
    }
}
```

```
package com.feel.agentcore.visualization;

import com.feel.agentcore.core.CognitiveLoop;
import com.feel.agentcore.mind.AgentMind;
import com.feel.agentcore.action.ActionType;
import javax.swing.*;
import javax.swing.border.TitledBorder;
import java.awt.*;
import java.awt.event.*;
import java.util.*;
import java.util.List;
import javax.swing.Timer;

public class AgentWindowUI extends JFrame {
    private CognitiveLoop loop;
    private Timer updateTimer;
    private boolean isRunning = true;

    private JPanel mainPanel;
    private JPanel statePanel;
    private JPanel graphPanel;
    private JPanel actionPanel;
    private JPanel controlPanel;
    private JPanel moodPanel;
    private JLabel timeLabel;
    private JLabel speedLabel;
    private JTextArea logArea;
    private JScrollPane logScroll;

    private LinkedList<Float> energyHistory = new LinkedList<>();
    private LinkedList<Float> boredomHistory = new
LinkedList<>();
```

```
   private LinkedList<Float> curiosityHistory = new
LinkedList<>();
   private LinkedList<Float> frustrationHistory = new
LinkedList<>();
   private LinkedList<Float> confidenceHistory = new
LinkedList<>();
   private final int MAX_HISTORY = 50;

   public AgentWindowUI() {
       this.loop = new CognitiveLoop();

       setupWindow();
       setupUI();
       setupControls();
       setupUpdateTimer();

       loop.start();
       setVisible(true);
   }

   private void setupWindow() {
       setTitle("🤖 Autonomous Agent - Living System");
       setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
       setSize(1200, 800);
       setLocationRelativeTo(null);

       addWindowListener(new WindowAdapter() {
           @Override
           public void windowClosing(WindowEvent e) {
               stopAgent();
           }
       });
   }

   private void setupUI() {
       mainPanel = new JPanel(new BorderLayout(10, 10));
       mainPanel.setBorder(BorderFactory.createEmptyBorder(10,
10, 10, 10));
       mainPanel.setBackground(new Color(240, 240, 245));

       createStatePanel();
```

```
        createGraphPanel();
        createActionPanel();
        createMoodPanel();
        createControlPanel();

        JPanel leftPanel = new JPanel(new BorderLayout(10, 10));
        leftPanel.setPreferredSize(new Dimension(300, 0));
        leftPanel.setBackground(new Color(240, 240, 245));
        leftPanel.add(statePanel, BorderLayout.NORTH);
        leftPanel.add(moodPanel, BorderLayout.CENTER);
        leftPanel.add(controlPanel, BorderLayout.SOUTH);

        JPanel rightPanel = new JPanel(new BorderLayout(10, 10));
        rightPanel.setBackground(new Color(240, 240, 245));
        rightPanel.add(graphPanel, BorderLayout.CENTER);
        rightPanel.add(actionPanel, BorderLayout.SOUTH);

        JSplitPane splitPane = new
JSplitPane(JSplitPane.HORIZONTAL_SPLIT, leftPanel, rightPanel);
        splitPane.setResizeWeight(0.25);

        mainPanel.add(splitPane, BorderLayout.CENTER);

        createLogPanel();
        mainPanel.add(logScroll, BorderLayout.SOUTH);

        add(mainPanel);
    }

    private void createStatePanel() {
        statePanel = new JPanel();
        statePanel.setLayout(new BoxLayout(statePanel,
BoxLayout.Y_AXIS));
        statePanel.setBorder(BorderFactory.createTitledBorder(
                BorderFactory.createLineBorder(new Color(100,
100, 150), 2),
                "🧠 Internal State",
                TitledBorder.LEFT,
                TitledBorder.TOP,
                new Font("Arial", Font.BOLD, 14)
        ));
```

```
        statePanel.setBackground(new Color(250, 250, 255));
        statePanel.setPreferredSize(new Dimension(280, 200));
    }

    private void createGraphPanel() {
        graphPanel = new JPanel() {
            @Override
            protected void paintComponent(Graphics g) {
                super.paintComponent(g);
                drawStateGraphs(g);
            }
        };
        graphPanel.setBorder(BorderFactory.createTitledBorder(
                BorderFactory.createLineBorder(new Color(100,
100, 150), 2),
                "📈 State History (Last 50 Ticks)",
                TitledBorder.LEFT,
                TitledBorder.TOP,
                new Font("Arial", Font.BOLD, 14)
        ));
        graphPanel.setBackground(Color.WHITE);
        graphPanel.setPreferredSize(new Dimension(0, 300));
    }

    private void createActionPanel() {
        actionPanel = new JPanel();
        actionPanel.setLayout(new BoxLayout(actionPanel,
BoxLayout.Y_AXIS));
        actionPanel.setBorder(BorderFactory.createTitledBorder(
                BorderFactory.createLineBorder(new Color(100,
100, 150), 2),
                "🎬 Recent Actions",
                TitledBorder.LEFT,
                TitledBorder.TOP,
                new Font("Arial", Font.BOLD, 14)
        ));
        actionPanel.setBackground(new Color(250, 250, 255));
        actionPanel.setPreferredSize(new Dimension(0, 150));
    }

    private void createMoodPanel() {
```

```
        moodPanel = new JPanel();
        moodPanel.setLayout(new BorderLayout());
        moodPanel.setBorder(BorderFactory.createTitledBorder(
                BorderFactory.createLineBorder(new Color(100,
100, 150), 2),
                "🎭 Current Mood",
                TitledBorder.LEFT,
                TitledBorder.TOP,
                new Font("Arial", Font.BOLD, 14)
        ));
        moodPanel.setBackground(new Color(250, 250, 255));
        moodPanel.setPreferredSize(new Dimension(280, 100));
    }

    private void createControlPanel() {
        controlPanel = new JPanel(new GridLayout(4, 2, 5, 5));
        controlPanel.setBorder(BorderFactory.createTitledBorder(
                BorderFactory.createLineBorder(new Color(100,
100, 150), 2),
                "🎮 Controls",
                TitledBorder.LEFT,
                TitledBorder.TOP,
                new Font("Arial", Font.BOLD, 14)
        ));
        controlPanel.setBackground(new Color(250, 250, 255));
        controlPanel.setPreferredSize(new Dimension(280, 150));

        controlPanel.add(new JLabel("⏱️ Time:"));
        timeLabel = new JLabel("00:00");
        timeLabel.setFont(new Font("Monospaced", Font.BOLD, 14));
        controlPanel.add(timeLabel);

        controlPanel.add(new JLabel("⚡ Speed:"));
        speedLabel = new JLabel("1x");
        speedLabel.setFont(new Font("Monospaced", Font.BOLD,
14));
        controlPanel.add(speedLabel);

        JButton pauseButton = new JButton("⏯️ Pause/Resume");
        pauseButton.addActionListener(e -> togglePause());
        controlPanel.add(pauseButton);
```

```
        JButton speed1Button = new JButton("1x Normal");
        speed1Button.addActionListener(e -> setSpeed(1.0f));
        controlPanel.add(speed1Button);

        JButton speed5Button = new JButton("5x Fast");
        speed5Button.addActionListener(e -> setSpeed(5.0f));
        controlPanel.add(speed5Button);

        JButton speed10Button = new JButton("10x Very Fast");
        speed10Button.addActionListener(e -> setSpeed(10.0f));
        controlPanel.add(speed10Button);

        JButton saveButton = new JButton("💾 Save State");
        saveButton.addActionListener(e -> saveState());
        controlPanel.add(saveButton);

        JButton resetButton = new JButton("🔄 Reset");
        resetButton.addActionListener(e -> resetAgent());
        controlPanel.add(resetButton);
    }

    private void createLogPanel() {
        logArea = new JTextArea(5, 0);
        logArea.setEditable(false);
        logArea.setFont(new Font("Monospaced", Font.PLAIN, 12));
        logScroll = new JScrollPane(logArea);
        logScroll.setBorder(BorderFactory.createTitledBorder(
                BorderFactory.createLineBorder(new Color(100,
100, 150), 2),
                "📝 Observation Log",
                TitledBorder.LEFT,
                TitledBorder.TOP,
                new Font("Arial", Font.BOLD, 14)
        ));
        logScroll.setPreferredSize(new Dimension(0, 120));
    }

    private void setupControls() {
        KeyboardFocusManager.getCurrentKeyboardFocusManager()
                .addKeyEventDispatcher(e -> {
```

```
                if (e.getID() == KeyEvent.KEY_PRESSED) {
                    switch (e.getKeyCode()) {
                        case KeyEvent.VK_SPACE:
                            togglePause();
                            return true;
                        case KeyEvent.VK_1:
                            setSpeed(1.0f);
                            return true;
                        case KeyEvent.VK_2:
                            setSpeed(5.0f);
                            return true;
                        case KeyEvent.VK_3:
                            setSpeed(10.0f);
                            return true;
                        case KeyEvent.VK_S:
                            saveState();
                            return true;
                    }
                }
                return false;
            });
}

private void setupUpdateTimer() {
    updateTimer = new Timer(200, e -> updateDisplay());
    updateTimer.start();
}

private void updateDisplay() {
    if (!isRunning) return;

    AgentMind mind = loop.getAgentMind();

    updateHistory(mind);
    updateStatePanel(mind);
    updateMoodPanel(mind);
    updateActionPanel(mind);
    updateGraphPanel();
    updateTimeAndSpeed();
    updateLog();
```

```
        revalidate();
        repaint();
    }

    private void updateHistory(AgentMind mind) {
        energyHistory.add(mind.getEnergy());
        boredomHistory.add(mind.getBoredom());
        curiosityHistory.add(mind.getCuriosity());
        frustrationHistory.add(mind.getFrustration());
        confidenceHistory.add(mind.getConfidence());

        if (energyHistory.size() > MAX_HISTORY) {
            energyHistory.removeFirst();
            boredomHistory.removeFirst();
            curiosityHistory.removeFirst();
            frustrationHistory.removeFirst();
            confidenceHistory.removeFirst();
        }
    }

    private void updateStatePanel(AgentMind mind) {
        statePanel.removeAll();

        statePanel.add(createStateBar("ENERGY", mind.getEnergy(),
new Color(50, 150, 50)));
        statePanel.add(createStateBar("BOREDOM",
mind.getBoredom(), new Color(150, 100, 50)));
        statePanel.add(createStateBar("CURIOSITY",
mind.getCuriosity(), new Color(50, 100, 150)));
        statePanel.add(createStateBar("FRUSTRATION",
mind.getFrustration(), new Color(200, 50, 50)));
        statePanel.add(createStateBar("CONFIDENCE",
mind.getConfidence(), new Color(150, 50, 150)));

        statePanel.revalidate();
    }

    private JPanel createStateBar(String label, float value,
Color color) {
        JPanel panel = new JPanel(new BorderLayout(5, 0));
        panel.setBackground(new Color(250, 250, 255));
```

```
        panel.setMaximumSize(new Dimension(Integer.MAX_VALUE,
25));

        JLabel nameLabel = new JLabel(label + ":");
        nameLabel.setPreferredSize(new Dimension(90, 20));

        JProgressBar bar = new JProgressBar(0, 100);
        bar.setValue((int)(value * 100));
        bar.setStringPainted(true);
        bar.setString(String.format("%3d%%", (int)(value *
100)));
        bar.setForeground(color);
        bar.setBackground(new Color(230, 230, 230));

        panel.add(nameLabel, BorderLayout.WEST);
        panel.add(bar, BorderLayout.CENTER);

        return panel;
    }

    private void updateMoodPanel(AgentMind mind) {
        moodPanel.removeAll();

        var mood = mind.getDerivedMood();

        JPanel content = new JPanel(new GridBagLayout());
        content.setBackground(new Color(250, 250, 255));

        GridBagConstraints gbc = new GridBagConstraints();
        gbc.gridx = 0;
        gbc.gridy = 0;
        gbc.insets = new Insets(5, 5, 5, 5);

        JLabel moodLabel = new
JLabel(mood.getInterpretation().toUpperCase());
        moodLabel.setFont(new Font("Arial", Font.BOLD, 18));
        moodLabel.setForeground(getMoodColor(mood.valence));
        content.add(moodLabel, gbc);

        gbc.gridy = 1;
        JPanel metricsPanel = new JPanel(new FlowLayout());
```

```
        metricsPanel.setBackground(new Color(250, 250, 255));
        metricsPanel.add(new JLabel("Valence: " +
formatFloat(mood.valence)));
        metricsPanel.add(new JLabel("Arousal: " +
formatFloat(mood.arousal)));
        content.add(metricsPanel, gbc);

        moodPanel.add(content, BorderLayout.CENTER);
        moodPanel.revalidate();
    }

    private void updateActionPanel(AgentMind mind) {
        actionPanel.removeAll();

        List<AgentMind.ActionRecord> recent =
mind.getRecentActionsList();

        if (recent.isEmpty()) {
            actionPanel.add(new JLabel("  No actions yet"));
        } else {
            int start = Math.max(0, recent.size() - 8);
            for (int i = start; i < recent.size(); i++) {
                AgentMind.ActionRecord record = recent.get(i);
                JPanel actionRow = new JPanel(new
FlowLayout(FlowLayout.LEFT));
                actionRow.setBackground(new Color(250, 250,
255));

                String arrow = i == recent.size() - 1 ? "▶ " : "
";
                float habitStrength =
mind.getHabitStrength(record.action);

                JLabel actionLabel = new JLabel(String.format("%s
%s (habit: %.2f)",
                        arrow, record.action, habitStrength));

                if (i == recent.size() - 1) {
                    actionLabel.setFont(new Font("Arial",
Font.BOLD, 12));
                }
```

```
                actionRow.add(actionLabel);
                actionPanel.add(actionRow);
            }
        }

        actionPanel.revalidate();
    }

    private void updateGraphPanel() {
        graphPanel.repaint();
    }

    private void drawStateGraphs(Graphics g) {
        if (energyHistory.isEmpty()) return;

        Graphics2D g2 = (Graphics2D) g;
        g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
RenderingHints.VALUE_ANTIALIAS_ON);

        int w = graphPanel.getWidth() - 40;
        int h = graphPanel.getHeight() - 40;
        int leftMargin = 30;
        int topMargin = 20;

        g2.setColor(new Color(220, 220, 220));
        for (int i = 0; i <= 5; i++) {
            int y = topMargin + (i * h / 5);
            g2.drawLine(leftMargin, y, leftMargin + w, y);
        }

        drawLine(g2, energyHistory, new Color(50, 150, 50),
leftMargin, topMargin, w, h);
        drawLine(g2, boredomHistory, new Color(150, 100, 50),
leftMargin, topMargin, w, h);
        drawLine(g2, curiosityHistory, new Color(50, 100, 150),
leftMargin, topMargin, w, h);
        drawLine(g2, frustrationHistory, new Color(200, 50, 50),
leftMargin, topMargin, w, h);
        drawLine(g2, confidenceHistory, new Color(150, 50, 150),
leftMargin, topMargin, w, h);
```

```
        g2.setFont(new Font("Arial", Font.PLAIN, 10));
        drawLegend(g2);
    }

    private void drawLine(Graphics2D g2, LinkedList<Float>
history, Color color,
                          int leftMargin, int topMargin, int w,
int h) {
        if (history.size() < 2) return;

        g2.setColor(color);
        g2.setStroke(new BasicStroke(2));

        int n = history.size();
        float dx = (float) w / (n - 1);

        int prevX = leftMargin;
        int prevY = topMargin + h - (int)(history.get(0) * h);

        for (int i = 1; i < n; i++) {
            int x = leftMargin + (int)(i * dx);
            int y = topMargin + h - (int)(history.get(i) * h);
            g2.drawLine(prevX, prevY, x, y);
            prevX = x;
            prevY = y;
        }
    }

    private void drawLegend(Graphics2D g2) {
        String[] names = {"Energy", "Boredom", "Curiosity",
"Frustration", "Confidence"};
        Color[] colors = {
                new Color(50, 150, 50),
                new Color(150, 100, 50),
                new Color(50, 100, 150),
                new Color(200, 50, 50),
                new Color(150, 50, 150)
        };

        int x = graphPanel.getWidth() - 120;
```

```
        int y = 30;

        for (int i = 0; i < names.length; i++) {
            g2.setColor(colors[i]);
            g2.fillRect(x, y + i * 15, 10, 10);
            g2.setColor(Color.BLACK);
            g2.drawString(names[i], x + 15, y + i * 15 + 8);
        }
    }

    private void updateTimeAndSpeed() {
        long seconds = System.currentTimeMillis() / 1000;
        timeLabel.setText(String.format("%02d:%02d", (seconds /
60) % 60, seconds % 60));

        float speed = loop.getTimeScale();
        speedLabel.setText(speed + "x");
    }

    private void updateLog() {
        if (Math.random() < 0.1) {
            AgentMind mind = loop.getAgentMind();
            String logEntry = String.format("[%s] E:%.2f B:%.2f
C:%.2f F:%.2f K:%.2f | Mood: %s\n",
                    timeLabel.getText(),
                    mind.getEnergy(), mind.getBoredom(),
mind.getCuriosity(),
                    mind.getFrustration(), mind.getConfidence(),
                    mind.getDerivedMood().getInterpretation());

            logArea.append(logEntry);
            String[] lines = logArea.getText().split("\n");
            if (lines.length > 20) {
                StringBuilder sb = new StringBuilder();
                for (int i = lines.length - 20; i < lines.length;
i++) {
                    sb.append(lines[i]).append("\n");
                }
                logArea.setText(sb.toString());
            }
        }
```

```
    }

    private Color getMoodColor(float valence) {
        if (valence > 0.3f) return new Color(0, 150, 0);
        if (valence < -0.3f) return new Color(200, 0, 0);
        return new Color(150, 150, 0);
    }

    private String formatFloat(float value) {
        return String.format("%+.2f", value);
    }

    private void togglePause() {
        loop.togglePause();
    }

    private void setSpeed(float speed) {
        loop.setTimeScale(speed);
    }

    private void saveState() {
        JOptionPane.showMessageDialog(this, "State saved to
saves/autosave.json");
    }

    private void resetAgent() {
        int result = JOptionPane.showConfirmDialog(this,
                "Reset agent to initial state?",
                "Confirm Reset",
                JOptionPane.YES_NO_OPTION);

        if (result == JOptionPane.YES_OPTION) {
            loop.stop();
            loop = new CognitiveLoop();
            loop.start();
            energyHistory.clear();
            boredomHistory.clear();
            curiosityHistory.clear();
            frustrationHistory.clear();
            confidenceHistory.clear();
            logArea.setText("");
```

```
        }
    }

    private void stopAgent() {
        updateTimer.stop();
        loop.stop();
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            try {

UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName
());
            } catch (Exception e) {
                e.printStackTrace();
            }
            new AgentWindowUI();
        });
    }
}
```

```
package com.feel.agentcore.world;

import com.feel.agentcore.action.Action;
import com.feel.agentcore.action.ActionType;
import com.feel.agentcore.mind.Feedback;
import java.util.*;
import java.util.concurrent.ConcurrentHashMap;

public class World {
    private List<Action> availableActions;
    private Map<ActionType, Float> actionCosts;
    private Map<ActionType, Integer> actionCounts;
    private Map<ActionType, Long> lastPerformedMap;
    private State currentState;

    private transient Random random;

    public World() {
```

```
        this.availableActions = Collections.synchronizedList(new
ArrayList<>());
        this.actionCosts = new ConcurrentHashMap<>();
        this.actionCounts = new ConcurrentHashMap<>();
        this.lastPerformedMap = new ConcurrentHashMap<>();
        this.random = new Random();

        initializeDefaultActions();
        this.currentState = new State(new
ArrayList<>(availableActions));
    }

    private void initializeDefaultActions() {
        addAction(new Action(ActionType.IDLE, 0.05f, 0.10f,
0.1f));
        addAction(new Action(ActionType.EXPLORE, 0.15f, -0.20f,
0.8f));
        addAction(new Action(ActionType.REPEAT_LAST, 0.08f,
-0.08f, 0.2f));
        addAction(new Action(ActionType.REST, -0.20f, 0.03f,
0.0f));
    }

    public void initAfterLoad() {
        if (random == null) {
            this.random = new Random();
        }
    }

    public void addAction(Action action) {
        availableActions.add(action);
        actionCosts.put(action.getType(),
action.getEnergyCost());
        actionCounts.putIfAbsent(action.getType(), 0);
        lastPerformedMap.putIfAbsent(action.getType(), 0L);
    }

    public void removeAction(ActionType type) {
        availableActions.removeIf(action -> action.getType() ==
type);
        actionCosts.remove(type);
```

```
        actionCounts.remove(type);
        lastPerformedMap.remove(type);
    }

    public void sense() {
        currentState = new State(new
ArrayList<>(availableActions));
    }

    public List<Action> getAvailableActions() {
        return new ArrayList<>(availableActions);
    }

    public Feedback executeAction(Action action) {
        if (action == null) return Feedback.NEUTRAL;

        actionCounts.merge(action.getType(), 1, Integer::sum);
        lastPerformedMap.put(action.getType(),
System.currentTimeMillis());

        return generateFeedback(action);
    }

    private Feedback generateFeedback(Action action) {
        float rand = random.nextFloat();

        switch (action.getType()) {
            case IDLE:
                // IDLE: mostly neutral or negative
                if (rand < 0.3f) return Feedback.POSITIVE;
                if (rand < 0.7f) return Feedback.NEUTRAL;
                return Feedback.NEGATIVE;

            case EXPLORE:
                // EXPLORE: balanced but generally positive
                if (rand < 0.5f) return Feedback.POSITIVE;
                if (rand < 0.8f) return Feedback.NEUTRAL;
                return Feedback.NEGATIVE;

            case REPEAT_LAST:
                // REPEAT: safe but not exciting
```

```
                if (rand < 0.3f) return Feedback.POSITIVE;
                if (rand < 0.8f) return Feedback.NEUTRAL;
                return Feedback.NEGATIVE;

            case REST:
                // REST: mostly positive
                if (rand < 0.6f) return Feedback.POSITIVE;
                if (rand < 0.9f) return Feedback.NEUTRAL;
                return Feedback.NEGATIVE;

            default:
                return Feedback.NEUTRAL;
        }
    }

    public Map<ActionType, Integer> getActionStatistics() {
        return new HashMap<>(actionCounts);
    }

    public long getLastPerformed(ActionType type) {
        return lastPerformedMap.getOrDefault(type, 0L);
    }

    public State getCurrentState() {
        return currentState;
    }

    public static class State {
        private final List<Action> availableActions;
        private final long timestamp;

        public State(List<Action> availableActions) {
            this.availableActions = availableActions;
            this.timestamp = System.currentTimeMillis();
        }

        public List<Action> getAvailableActions() { return
availableActions; }
        public long getTimestamp() { return timestamp; }
    }
}
```

```java
package com.feel.agentcore;

import com.feel.agentcore.visualization.AgentWindowUI;

import javax.swing.SwingUtilities;

public class Main {
   public static void main(String[] args) {
       SwingUtilities.invokeLater(() -> {
           new AgentWindowUI();
       });
   }
}
```